

# GIT & GITHUB — COMPLETE HANDWRITTEN NOTES

## 1. What is Git?

**Git = Distributed Version Control System (DVCS)**

Git is used to:

- Track changes in source code
- Maintain different versions of a project
- Go back to previous versions
- Work with multiple developers
- Create branches for different features
- Merge changes
- Collaborate with remote repositories such as GitHub

### Why do we need Git?

Without Git:

project  
project-final  
project-final-2  
project-final-new  
project-final-new-real  
project-final-latest

With Git:

One project

↓

Git tracks every version

↓

Can go back to any required version

**Important:** Git is a version-control tool.

GitHub is a platform for hosting Git repositories and collaborating.

---

## 2. Git vs GitHub

## Git

- Software/tool
- Installed on your computer
- Tracks changes
- Works locally
- Does not require internet for most operations

## GitHub

- Cloud-based platform
- Hosts Git repositories
- Enables collaboration
- Used for Pull Requests, Issues, Code Reviews, etc.

## Simple example

Git = Version Control System

GitHub = Online platform for Git repositories

---

# 3. Git Installation

After installing Git, verify:

```
git --version
```

Example:

```
git version 2.x.x
```

Configure your identity:

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

Check configuration:

```
git config --list
```

## Important

Git associates commits with:

Name

Email

So configure these before making commits.

---

## 4. Git Repository

A **repository (repo)** is a location where Git tracks your project.

There are two major types:

Local Repository

Remote Repository

### Local Repository

Exists on your computer.

### Remote Repository

Exists on a remote server such as GitHub.

---

## 5. Working Directory

The **working directory** is where you actually work on your project.

Example:

```
my-project/  
|  
├── index.html  
├── style.css  
└── script.js
```

When you modify these files, Git detects the changes.

---

## 6. Git's Three Important Areas

This is **VERY IMPORTANT**.

Remember:

Working Directory



Staging Area  
↓  
Local Repository  
↓  
Remote Repository

## 1. Working Directory

Where you create/change files.

## 2. Staging Area

Files selected for the next commit.

## 3. Repository

Committed versions are stored here.

## 4. Remote Repository

Repository hosted somewhere like GitHub.

---

# 7. Initialize a Git Repository

Go inside your project:

```
cd my-project
```

Initialize Git:

```
git init
```

Git creates:

```
.git/
```

The `.git` directory contains Git's internal repository information.

## Important

Do NOT manually modify `.git` unless you know what you're doing.

---

## 8. Check Git Status

Command:

```
git status
```

Used to see:

- Modified files
- Untracked files
- Staged files
- Current branch

Example:

Untracked files:  
index.html

---

## 9. Git Add

To stage a file:

```
git add index.html
```

Stage all files:

```
git add .
```

or:

```
git add -A
```

Flow:

Working Directory



git add



Staging Area

### Important

**git add** does NOT create a commit.

It only stages the changes.

---

## 10. Git Commit

Commit staged changes:

```
git commit -m "Initial commit"
```

Example:

```
git commit -m "Add login page"
```

A commit is a **snapshot of your project at a particular point in time**.

Flow:

Working Directory



git add



Staging Area



git commit



Local Repository

### Good commit messages

```
git commit -m "Add user authentication"
```

Bad:

```
git commit -m "changes"
```

Commit messages should explain what changed.

---

## 11. Git Log

View previous commits:

```
git log
```

Short version:

```
git log --oneline
```

Example:

a83f21d Add login page  
7bc91aa Add homepage  
21de982 Initial commit

Each commit has a unique **commit hash**.

---

## 12. Git Diff

Check changes that haven't been staged:

`git diff`

Check staged changes:

`git diff --staged`

Useful for reviewing your changes before committing.

---

## 13. Untracked vs Modified vs Staged

### Untracked

Git doesn't know about the file yet.

file.txt

### Modified

Git knows the file but its content changed.

Modified: file.txt

### Staged

You used:

`git add file.txt`

and Git has selected it for the next commit.

---

## 14. .gitignore

`.gitignore` tells Git which files/directories **NOT** to track.

Example:

```
node_modules/  
.env  
*.log  
__pycache__/
```

Typical things to ignore:

- Passwords
- API keys
- Environment variables
- Logs
- Dependencies
- Temporary files
- Build files

### Example

```
.env
```

Never accidentally push secrets to GitHub.

---

## 15. GitHub Repository

Create a repository on GitHub.

Then connect your local repository to GitHub.

Check remotes:

```
git remote -v
```

Add remote:

```
git remote add origin <repository-url>
```

Example concept:

Local Git Repository

↓  
origin

↓  
GitHub Repository

`origin` is the conventional name for the main remote repository.

---

## 16. Git Push

Send local commits to GitHub:

```
git push origin main
```

Often:

```
git push -u origin main
```

`-u` establishes the upstream relationship so future pushes can often simply use:

```
git push
```

### Flow

Local Repository  
↓  
git push  
↓  
GitHub Repository

---

## 17. Git Clone

To download an existing repository:

```
git clone <repository-url>
```

Example:

```
git clone https://github.com/user/project.git
```

Clone does more than just download files:

Remote repository  
↓  
git clone  
↓  
Local repository

+  
Working directory  
+  
Git history

---

## 18. Git Fetch vs Git Pull

### git fetch

Downloads changes from remote but **does not automatically merge them** into your current branch.

git fetch

Think:

GitHub → Local information

### git pull

Gets remote changes and integrates them into your current branch.

git pull

Conceptually:

git pull  $\approx$  git fetch + git merge

### Remember

fetch = download/check

pull = download + integrate

---

## 19. Branches

A branch allows you to work on a separate line of development.

Example:

```
main
|
|— feature-login
|
|— feature-payment
```

|  
└─ bugfix-navbar

## Why branches?

Suppose:

main = production code

You want to add login.

Instead of directly changing **main**:

main

↓

feature/login

Develop the feature there.

---

## 20. Check Current Branch

git branch

Current branch is usually marked with:

\*

Example:

\* main

feature-login

---

## 21. Create a Branch

git branch feature-login

This creates the branch but doesn't switch to it.

Create + switch:

git checkout -b feature-login

Modern command:

git switch -c feature-login

---

## 22. Switch Branch

Older command:

```
git checkout main
```

Modern command:

```
git switch main
```

---

## 23. Delete Branch

```
git branch -d feature-login
```

Force delete:

```
git branch -D feature-login
```

Use `-D` carefully because it can delete an unmerged branch.

---

## 24. Branch Workflow

Typical workflow:

main

↓

Create feature branch

↓

Develop feature

↓

git add

↓

git commit

↓

Push branch

↓

Create Pull Request

↓

Code Review

↓

Merge into main

This is extremely important in professional development.

---

## 25. Merge

Suppose:

```
main
 \
  feature-login
```

After finishing the feature, merge it into `main`.

First:

```
git switch main
```

Then:

```
git merge feature-login
```

Concept:

```
feature-login
  ↓
  merge
  ↓
  main
```

---

## 26. Fast-Forward Merge

If `main` hasn't changed since the feature branch was created, Git may simply move the branch pointer forward.

Concept:

```
A---B---C
    ↑
  feature
```

main → B

After merge:

```
A---B---C
      ↑
    main/feature
```

No additional merge commit is required.

---

## 27. Merge Commit

If both branches have different commits:

```
      C---D feature
      /
A---B
      \
      E---F main
```

Git may create a merge commit:

```
      C---D
      /  \
A---B      M
      \  /
      E---F
```

M = merge commit.

---

## 28. Merge Conflicts

A merge conflict happens when Git cannot automatically determine which change should be kept.

Example:

Branch A:

Hello World

Branch B:

Hello Git

Both modify the same part of the file.

Git may produce:

```
<<<<<< HEAD
Hello World
=====
Hello Git
>>>>>> feature
```

You must manually decide what the final content should be.

Then:

```
git add .
```

and:

```
git commit
```

### **Conflict resolution process**

```
Merge
↓
Conflict
↓
Open file
↓
Choose/fix content
↓
Remove conflict markers
↓
git add
↓
git commit
```

---

## **29. Git Merge vs Git Rebase**

Both can integrate changes, but they work differently.

### **Merge**

Preserves branch history.

```
    C---D
   /   \
  A---B   M
```

\ /  
E---F

## Rebase

Moves/replays your commits on top of another branch.

Before:

    C---D feature  
    /  
A---B---E---F main

After rebase:

A---B---E---F---C'---D'

Your commits are replayed on top of the updated base.

---

## 30. Important Rebase Warning

Rebase **rewrites commit history**.

Therefore:

Avoid rebasing commits that other people are already depending on/shared branches unless you understand the consequences.

For local/private work, rebase is commonly useful.

---

## 31. Interactive Rebase

Command:

git rebase -i HEAD~3

Used to modify recent commits.

Can be used to:

- Reorder commits
- Squash commits
- Edit commit messages

- Edit commits

Common keywords:

pick  
reword  
edit  
squash  
fixup  
drop

## Squash

Combine multiple commits into one.

Example:

Add login  
Fix login  
Fix login again

can become:

Add login feature

---

## 32. Git Stash

Suppose you're working on something:

feature-A

Your changes are unfinished.

Suddenly someone asks you to switch to:

feature-B

You don't want to commit incomplete work.

Use:

git stash

Your uncommitted changes are temporarily stored.

Now switch branches:

git switch feature-B

Do your work.

Return:

`git switch feature-A`

Restore changes:

`git stash pop`

---

## 33. Git Stash Commands

Save changes:

`git stash`

List stashes:

`git stash list`

Apply latest stash:

`git stash apply`

Apply and remove from stash:

`git stash pop`

Delete stash:

`git stash drop`

Clear all stashes:

`git stash clear`

### Difference

`stash apply` → restores + keeps stash

`stash pop` → restores + removes stash

---

## 34. Git Reset

`git reset` moves the current branch/HEAD to another commit and can also change the staging/working state depending on the option.

Three important modes:

--soft  
--mixed  
--hard

## Soft

git reset --soft HEAD~1

Moves HEAD back but keeps changes staged.

## Mixed

git reset HEAD~1

(or --mixed)

Moves HEAD back and unstages changes, while keeping file modifications.

## Hard

git reset --hard HEAD~1

Moves HEAD back and discards changes in tracked files.

⚠ Be careful with --hard.

---

# 35. HEAD

HEAD represents your current position in Git history—normally the current commit/branch you're on.

Example:

A---B---C  
    ↑  
  HEAD

Previous commit:

HEAD~1

Two commits back:

HEAD~2

---

## 36. Git Revert

`git revert` creates a new commit that reverses the effect of an earlier commit.

Example:

A---B---C

Revert C:

A---B---C---D

D undoes the changes introduced by C.

Command:

```
git revert <commit-hash>
```

### Reset vs Revert

#### Reset:

Moves history pointer

#### Revert:

Creates a new commit that undoes previous changes

For shared/public history, `revert` is generally safer because it doesn't rewrite existing history.

---

## 37. GitHub Pull Request (PR)

A **Pull Request** is a request to merge changes from one branch into another.

Typical workflow:

Create branch



Write code



Commit



Push branch to GitHub



Create Pull Request  
↓  
Code review  
↓  
Changes requested / approved  
↓  
Merge

---

## 38. Pull Request Example

Suppose:

main  
feature/login

You developed:

feature/login

Push it:

```
git push -u origin feature/login
```

Then GitHub allows you to create:

feature/login → main

Pull Request.

---

## 39. Pull Request Benefits

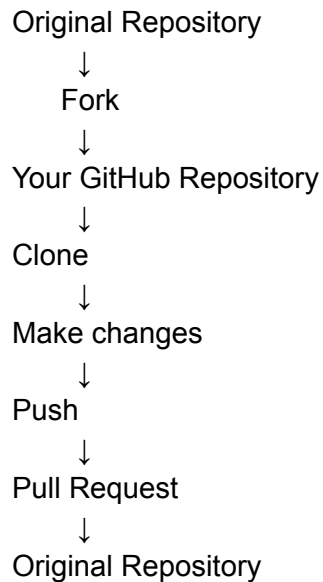
PRs allow teams to:

- Review code
  - Discuss changes
  - Request modifications
  - Run automated tests/checks
  - Maintain code quality
  - Collaborate before merging
-

## 40. Fork

A **fork** creates your own copy of someone else's GitHub repository under your account.

Typical open-source workflow:



## 41. Remote

A remote is a reference to another repository.

View:

```
git remote -v
```

Add:

```
git remote add origin <url>
```

Change URL:

```
git remote set-url origin <url>
```

---

## 42. Origin

**origin** is simply the conventional default name for the remote you cloned from or added.

Example:

git push origin main

Means:

Push branch "main"  
to remote "origin"

---

## 43. Git Command Cheat Sheet

### Repository

git init  
git clone <url>

### Configuration

git config --global user.name "Name"  
git config --global user.email "email"  
git config --list

### Status

git status

### Stage

git add file  
git add .

### Commit

git commit -m "message"

### History

git log  
git log --oneline

### Changes

git diff  
git diff --staged

### Branches

git branch  
git branch <name>  
git switch <name>

git switch -c <name>  
git branch -d <name>

## **Merge**

git merge <branch>

## **Remote**

git remote -v  
git remote add origin <url>

## **Push**

git push  
git push origin main

## **Pull**

git pull

## **Fetch**

git fetch

## **Clone**

git clone <url>

## **Stash**

git stash  
git stash list  
git stash apply  
git stash pop  
git stash drop  
git stash clear

## **Reset**

git reset --soft HEAD~1  
git reset HEAD~1  
git reset --hard HEAD~1

## **Revert**

git revert <commit>

## **Rebase**

git rebase <branch>

git rebase -i HEAD~3

---

## 44. MOST IMPORTANT WORKFLOW TO MEMORIZE

For your DevOps/cloud career, **write this in a box in your notebook**:

CREATE PROJECT

↓  
git init  
↓  
git add .  
↓  
git commit -m "message"  
↓  
git remote add origin <URL>  
↓  
git push -u origin main

For daily development:

Modify code

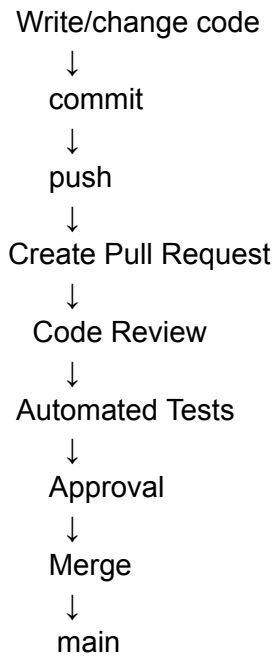
↓  
git status  
↓  
git add .  
↓  
git commit -m "message"  
↓  
git push

---

## 45. PROFESSIONAL TEAM WORKFLOW

This is especially important for your DevOps preparation.

main  
|  
Create feature branch  
↓  
feature/my-feature  
↓



## 46. Commands You Should Actually Practice

Don't just copy these. **Open your terminal and perform them.**

### Day 1 — Basics

```
mkdir git-practice  
cd git-practice
```

```
git init
```

```
touch README.md
```

```
git status
```

```
git add README.md
```

```
git commit -m "Initial commit"
```

```
git log --oneline
```

### Day 2 — GitHub

```
git remote add origin <github-url>
```

```
git branch -M main
```

```
git push -u origin main
```

### **Day 3 — Branching**

```
git switch -c feature-login
```

```
touch login.txt
```

```
git add .
```

```
git commit -m "Add login feature"
```

```
git push -u origin feature-login
```

### **Day 4 — Merge**

```
git switch main
```

```
git merge feature-login
```

### **Day 5 — Stash**

```
git switch -c test
```

```
# make changes
```

```
git stash
```

```
git stash list
```

```
git stash pop
```

### **Day 6 — Rebase/Revert/Reset**

Practice:

```
git reset --soft HEAD~1
```

```
git reset HEAD~1
```

```
git revert <commit>
```

```
git rebase main
```

**Do these in a practice repository**, not your important projects.

---

## **47. VERY IMPORTANT DIFFERENCES**

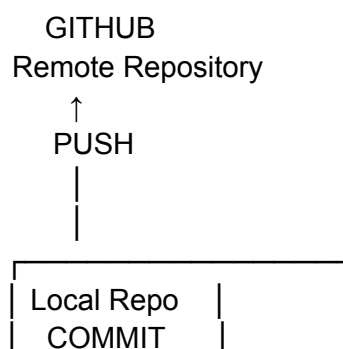
Write this table in your notes:

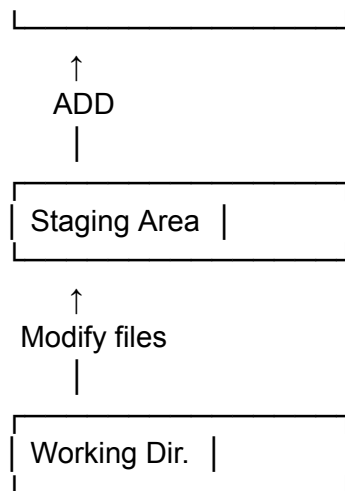
| Command                 | Purpose                                                 |
|-------------------------|---------------------------------------------------------|
| <code>git add</code>    | Working directory → staging                             |
| <code>git commit</code> | Staging → local repository                              |
| <code>git push</code>   | Local → remote                                          |
| <code>git fetch</code>  | Get remote updates without integrating                  |
| <code>git pull</code>   | Fetch + integrate                                       |
| <code>git clone</code>  | Copy remote repository locally                          |
| <code>git merge</code>  | Combine branch histories                                |
| <code>git rebase</code> | Replay commits onto another base                        |
| <code>git stash</code>  | Temporarily save uncommitted work                       |
| <code>git reset</code>  | Move branch/HEAD and potentially alter staging/worktree |
| <code>git revert</code> | Create commit that undoes another commit                |

---

## 48. THE ONE DIAGRAM TO REMEMBER

Put this at the end of your handwritten notes:





And:

REMOTE → LOCAL



git fetch



Download changes only

REMOTE → LOCAL + INTEGRATE



git pull